

Memoria del Proyecto DRM

Curso 2025–2026

Contenido

1. Introduccion.....	2
2. Objetivos del Proyecto.....	2
3. Arquitectura General del Sistema.....	2
4. Desarrollo del Sistema.....	3
4.1 Parte I – Comunicacion y Cifrado Basico.....	3
4.2 Parte II – Marca de Agua y Firma Digital.....	3
4.3 Parte III – Separacion UA/CDM.....	4
5. Codigos.....	4
6. Componentes del Sistema (en orden de ejecucion).....	21
7. Capturas de Funcionamiento.....	21
8. Pruebas y Validacion.....	23
9. Reparto de Tareas del Grupo.....	24
10. Conclusiones.....	25

1. Introducción

El presente proyecto implementa un sistema de Gestion de Derechos Digitales (DRM) basado en los principios empleados por sistemas reales como Widevine, PlayReady o FairPlay.

El objetivo es proteger contenidos digitales (imágenes y vídeos) mediante cifrado simétrico, licenciamiento seguro, firmas digitales y un módulo especializado de descifrado (CDM).

El sistema desarrollado consta de:

- Un servidor de contenidos.
- Un servidor de licencias.
- Una aplicación de usuario (UA).
- Un módulo de descifrado (CDM).

Estos componentes interactúan para permitir la distribución segura de contenidos cifrados y la obtención controlada de claves de descifrado.

2. Objetivos del Proyecto

- Implementar un flujo DRM completo basado en cifrado AES y RSA.
- Proteger imágenes y vídeos mediante cifrado CTR o CBC.
- Añadir marca de agua visible a las imágenes.
- Establecer claves de sesión mediante RSA y canal seguro UA-Servidor de licencias.
- Separar la lógica criptográfica sensible dentro del módulo de descifrado (CDM).
- Validar solicitudes mediante firma digital RSA.
- Cumplir las partes I, II y III del documento oficial de la asignatura.

3. Arquitectura General del Sistema

El sistema se compone de cuatro entidades principales:

- 1) Servidor de Contenidos: alberga los archivos cifrados y genera manifiestos individuales.
- 2) Aplicación de Usuario (UA): solicita contenidos, gestiona manifiestos y coordina la petición de licencias.
- 3) Servidor de Licencias: almacena las claves AES cifradas y entrega licencias tras validar firmas y solicitudes.
- 4) Módulo de Descifrado (CDM): firma solicitudes y descifra las licencias, devolviendo la clave AES en claro solo a la UA.

Flujo resumido:

UA → LIST/GET → Servidor de Contenidos

UA → CDM: firma de solicitud

UA → Servidor de Licencias: petición cifrada

Servidor → UA → CDM: entrega de licencia cifrada

CDM → UA: clave AES descifrada
UA descifra el contenido y lo guarda en “descifrados/”.

4. Desarrollo del Sistema

4.1 Parte I – Comunicación y Cifrado Básico

En esta fase se implementaron los elementos base:

Comando LIST para obtener la lista de archivos.
Comando GET para solicitar un fichero.
Generación del manifiesto adjunto al contenido.
Cifrado AES-CTR de los contenidos.
Descarga y almacenado en la UA.

El servidor de contenidos (servidor_contenidos.py) usa AES (CTR) con claves e IV definidos en contenido_informacion.json.

La UA interpreta el manifiesto y, si el contenido no está cifrado, lo descarga directamente.

4.2 Parte II – Marca de Agua y Firma Digital

En esta fase se introducen dos ampliaciones:

Marca de agua visible en imágenes

El servidor añade una marca con el formato “UA-XXXX” dependiendo del usuario que realiza la petición.

Firma digital RSA

Las solicitudes de licencia deben ir firmadas.
El CDM firma la estructura JSON que contiene el key_id y el client_id usando rsa_sign_with_pow().

El servidor de licencias valida la firma mediante rsa_verify_with_pow().

4.3 Parte III – Separación UA/CDM

La aplicación se dividio en dos módulos:

- UA: solo orquesta la comunicación.
- CDM: contiene funciones criptográficas sensibles.

El flujo:

1. La UA detecta contenido cifrado.
2. Pide al CDM que firme la solicitud.
3. Envía la solicitud firmada al servidor de licencias.
4. Recibe licencia cifrada AES-GCM.
5. La reenvía al CDM, que la descifra usando RSA.
6. Devuelve la clave AES a la UA, que descifra el contenido recibido del servidor de contenidos.

Esto garantiza separación de responsabilidades igual que en sistemas DRM reales.

5. CÓDIGOS

• generar_claves.py — crea claves RSA para servidor y CDM.

```
1 from utilidades_criptografia import generar_par_rsa, guardar_rsa_privada, guardar_rsa_publica
2 import os
3
4 os.makedirs('claves', exist_ok=True)
5
6 # Entidades para las que se generarán las claves
7 pares = {
8     'servidor_licencias': None,
9     'modulo_descifrado': None,
10 }
11
12 # Generación y guardado de las claves RSA
13 for nombre in pares:
14     clave_privada, clave_publica = generar_par_rsa()
15     guardar_rsa_privada(clave_privada, f'claves/{nombre}_privada.pem')
16     guardar_rsa_publica(clave_publica, f'claves/{nombre}_publica.pem')
17     print(f"Claves RSA generadas para '{nombre}' en claves/{nombre}_*.pem")
```

Generación y guardado de las claves RSA

Este fragmento es el núcleo del programa porque realiza la generación y el almacenamiento de las claves RSA.

El bucle recorre cada entidad definida en `pares`. Para cada una, se genera un par de claves RSA (privada y pública) mediante la función `generar_par_rsa()`.

• generar_base_licencias.py — genera base cifrada con claves AES.

```

1 import os, json, base64
2 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
3
4 ARCHIVO_CLAVE = 'claves/base_licencias_clave.bin'
5 ARCHIVO_CIFRADO = 'base_licencias_cifrada.enc'
6
7 os.makedirs('claves', exist_ok=True)
8
9 # Crear o cargar la clave maestra
10 if not os.path.exists(ARCHIVO_CLAVE):
11     clave = AESGCM.generate_key(bit_length=128)
12     with open(ARCHIVO_CLAVE, 'wb') as f:
13         f.write(clave)
14     print(f"[generar_base_licencias] Clave maestra creada en {ARCHIVO_CLAVE}")
15 else:
16     with open(ARCHIVO_CLAVE, 'rb') as f:
17         clave = f.read()
18     print(f"[generar_base_licencias] Clave maestra ya existente, se reutiliza.")
19
20 # Base de datos de licencias
21 base_licencias = {
22     "key_la_falete": "zw3VaJP256Dgcaw96BzX1g==",
23     "key_rioja": "dK5xPzQ7w9kK89jsA1X9tg==",
24     "key_torrente": "8XfZ5kD1yWqzP7lYfD3sRw==",
25     "key_remontada": "A8s9DfK6rTy4Q1vH7nOp0Q=="
26 }
27
28 # Cifrar la base de datos
29 aesgcm = AESGCM(clave)
30 nonce = os.urandom(12)
31 texto_plano = json.dumps(base_licencias).encode()
32 texto_cifrado = aesgcm.encrypt(nonce, texto_plano, None)
33
34 # Guardar resultado
35 resultado = {
36     "nonce": base64.b64encode(nonce).decode(),
37     "ciphertext": base64.b64encode(texto_cifrado).decode()
38 }
39
40 with open(ARCHIVO_CIFRADO, 'w', encoding='utf-8') as f:
41     json.dump(resultado, f, indent=4)
42
43 print("[generar_base_licencias] Base de datos cifrada creada correctamente.")

```

Cifrar la base de datos

Este fragmento es el más significativo porque realiza el cifrado real de la base de datos de licencias.

Primero se crea un objeto `AESGCM` utilizando la clave maestra previamente generada o cargada. A continuación se genera un `nonce` aleatorio de 12 bytes, que es necesario para garantizar la seguridad del cifrado y evitar repeticiones.

• cifrar_contenido.py

```

1 import os, base64, json, io
2 from PIL import Image, ImageDraw, ImageFont
3 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
4
5 ARCHIVO_INFO = "contenido_informacion.json"
6 CARPETA_ORIGEN = "contenido"
7 CARPETA_SALIDA = "contenido_cifrado"
8
9 # Crear carpeta de salida si no existe
10 os.makedirs(CARPETA_SALIDA, exist_ok=True)
11
12 # Cargar manifiesto de contenidos
13 with open(ARCHIVO_INFO, "r", encoding="utf-8") as f:
14     info = json.load(f)
15
16 def añadir_marca_agua(ruta_imagen, texto_marca="UA-STATIC"): # Añade una marca de agua visible a una imagen
17     imagen = Image.open(ruta_imagen).convert("RGBA")
18     ancho, alto = imagen.size
19
20     capa_marca = Image.new("RGBA", imagen.size, (0, 0, 0, 0))
21     dibujar = ImageDraw.Draw(capa_marca)
22
23     tamaño_fuente = max(12, int(ancho / 15))
24
25     try:
26         fuente = ImageFont.truetype("arial.ttf", tamaño_fuente)
27     except:
28         fuente = ImageFont.load_default()
29
30     try:
31         bbox = dibujar.textbbox((0, 0), texto_marca, font=fuente)
32         ancho_texto = bbox[2] - bbox[0]
33         alto_texto = bbox[3] - bbox[1]
34     except AttributeError:
35         ancho_texto, alto_texto = dibujar.textsize(texto_marca, font=fuente)
36
37     x = ancho - ancho_texto - 10
38     y = alto - alto_texto - 10
39
40     dibujar.text((x, y), texto_marca, font=fuente, fill=(255, 255, 255, 128))
41
42     imagen_marcada = Image.alpha_composite(imagen, capa_marca)
43
44     buffer = io.BytesIO()
45     imagen_marcada.convert("RGB").save(buffer, format="JPEG")
46

```

```

46     return buffer.getvalue()
47
48
49 for nombre_fichero, datos in info.items():
50     ruta_entrada = os.path.join(CARPETA_ORIGEN, nombre_fichero)
51     ruta_salida = os.path.join(CARPETA_SALIDA, nombre_fichero)
52
53     if not os.path.exists(ruta_entrada):
54         print(f"No existe el archivo: {ruta_entrada}")
55         continue
56
57     clave = base64.b64decode(datos["key"])
58
59     iv_b64 = datos["iv"]
60     if iv_b64.startswith("b64:"):
61         iv_b64 = iv_b64[4:]
62     iv = base64.b64decode(iv_b64 + "===")
63
64     # Si es imagen → añadir marca de agua
65     extension = os.path.splitext(nombre_fichero)[1].lower()
66     if extension in [".jpg", ".jpeg", ".png"]:
67         print(f"Añadiendo marca de agua a {nombre_fichero}...")
68         bytes_marcados = añadir_marca_agua(ruta_entrada)
69     else:
70         with open(ruta_entrada, "rb") as f:
71             bytes_marcados = f.read()
72
73     cifrador = Cipher(algorithms.AES(clave), modes.CTR(iv))
74     encriptador = cifrador.encryptor()
75     contenido_cifrado = encriptador.update(bytes_marcados) + encriptador.finalize()
76
77     with open(ruta_salida, "wb") as f:
78         f.write(contenido_cifrado)
79
80     print(f"'{nombre_fichero}' cifrado correctamente y guardado en {ruta_salida}")

```

Si es imagen → añadir marca de agua

Este fragmento es la parte más significativa porque realiza el cifrado efectivo del contenido antes de guardarlo.

Se crea un objeto **Cipher** usando el algoritmo AES en modo CTR, empleando una clave secreta y un vector de inicialización (**iv**). A continuación se obtiene el objeto **encryptor**, que se encarga de transformar los datos originales (ya sea una imagen con marca de agua o cualquier otro archivo) en datos cifrados.

- **contenido_servidor.py**

```

1 import socket, os, json, io, base64
2 from PIL import Image, ImageDraw, ImageFont
3 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
4
5 HOST = 'localhost'
6 PORT = 5000
7
8 CARPETA_ORIGEN = 'contenido'
9 ARCHIVO_INFO = 'contenido_informacion.json'
10
11 def añadir_marca_agua_bytes(imagen_bytes, texto_marca="UA-XXXX"): # Recibe bytes y devuelve bytes con marca de agua
12     try:
13         imagen = Image.open(io.BytesIO(imagen_bytes)).convert("RGBA")
14         ancho, alto = imagen.size
15
16         capa = Image.new("RGBA", imagen.size, (0, 0, 0, 0))
17         dibujar = ImageDraw.Draw(capa)
18         tamaño_fuente = max(12, int(ancho / 15))
19
20         try:
21             fuente = ImageFont.truetype("arial.ttf", tamaño_fuente)
22         except:
23             fuente = ImageFont.load_default()
24
25         try:
26             bbox = dibujar.textbbox((0, 0), texto_marca, font=fuente)
27             ancho_texto = bbox[2] - bbox[0]
28             alto_texto = bbox[3] - bbox[1]
29         except AttributeError:
30             ancho_texto, alto_texto = dibujar.textsize(texto_marca, font=fuente)
31
32         x = ancho - ancho_texto - 10
33         y = alto - alto_texto - 10
34
35         dibujar.text((x, y), texto_marca, font=fuente, fill=(255, 255, 255, 128))
36
37         marcada = Image.alpha_composite(imagen, capa)
38         buffer = io.BytesIO()
39         marcada.convert("RGB").save(buffer, format="JPEG")
40         return buffer.getvalue()
41
42     except Exception as e:
43         print("[servidor_contenido] Error marca de agua:", e)
44         return imagen_bytes
45
46
47 def cifrar_aes_ctr(clave_bytes, iv_bytes, datos_bytes):
48     cifrador = Cipher(algorithms.AES(clave_bytes), modes.CTR(iv_bytes))
49     encriptador = cifrador.encryptor()
50     return encriptador.update(datos_bytes) + encriptador.finalize()
51
52
53 def manejar_cliente(conn, addr):
54     try:
55         raw = conn.recv(8192)
56         if not raw:
57             conn.close()
58         return

```



```

60 partes = raw.decode(errors='ignore').strip().split()
61 if len(partes) == 0:
62     conn.close()
63     return
64
65 comando = partes[0].upper()
66 etiqueta_usuario = None
67
68 if comando == 'LIST':
69     if not os.path.exists(CARPETA_ORIGEN):
70         os.makedirs(CARPETA_ORIGEN, exist_ok=True)
71
72     archivos = [
73         f for f in os.listdir(CARPETA_ORIGEN)
74         if os.path.isfile(os.path.join(CARPETA_ORIGEN, f))
75     ]
76     for f in archivos:
77         conn.sendall(f"{f}\n".encode())
78     conn.sendall(b"END\n")
79     return
80
81 if comando == 'GET':
82     if len(partes) < 2:
83         conn.sendall(b"ERROR: Falta el nombre del archivo\n")
84         return
85
86     nombre_archivo = partes[1]
87
88     etiqueta_usuario = partes[2] if len(partes) >= 3 else f"UA-{{addr[1]}}"
89
90     ruta_archivo = os.path.join(CARPETA_ORIGEN, nombre_archivo)
91     if not os.path.exists(ruta_archivo):
92         conn.sendall(b"ERROR: Archivo no encontrado\n")
93         return
94
95     if not os.path.exists(ARCHIVO_INFO):
96         conn.sendall(b"ERROR: Falta contenido_informacion.json\n")
97         return
98
99     info = json.load(open(ARCHIVO_INFO, "r", encoding="utf-8"))
100     manifiesto = info.get(nombre_archivo)
101     if not manifiesto:
102         conn.sendall(b"ERROR: Manifiesto no encontrado\n")
103         return
104
105     conn.sendall(f"MANIFEST {json.dumps(manifiesto)}\n".encode())
106
107     with open(ruta_archivo, "rb") as f:
108         datos_originales = f.read()
109
110     if nombre_archivo.lower().endswith((".jpg", ".jpeg", ".png")):
111         datos_marcados = añadir_marca_agua_bytes(datos_originales, texto_marca=etiqueta_usuario)
112     else:
113         datos_marcados = datos_originales
114
115     clave_b64 = manifiesto.get("key")
116     iv_raw = manifiesto.get("iv", "")
117     iv_b64 = iv_raw[4:] if iv_raw.startswith("b64:") else iv_raw

```

```

118
119
120         try:
121             clave_bytes = base64.b64decode(clave_b64)
122             iv_bytes = base64.b64decode(iv_b64)
123         except Exception as e:
124             conn.sendall(b'{"error": "clave o iv incorrectos"}')
125             print("[servidor_contenido] Error clave/iv:", e)
126             return
127
128         cifrado = cifrar_aes_ctr(clave_bytes, iv_bytes, datos_marcados)
129
130         conn.sendall(f"SIZE {len(cifrado)}\n".encode())
131         conn.sendall(cifrado)
132
133         print(f"[servidor_contenido] Enviado {nombre_archivo} a {etiqueta_usuario}")
134         return
135
136         conn.sendall(b"ERROR: Comando desconocido\n")
137
138     except Exception as e:
139         print("[servidor_contenido] Excepción:", e)
140     finally:
141         conn.close()
142
143 def main():
144     os.makedirs(CARPETA_ORIGEN, exist_ok=True)
145     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
146     s.bind((HOST, PORT))
147     s.listen(5)
148     print(f"[servidor_contenido] Escuchando en {HOST}:{PORT}")
149
150     try:
151         while True:
152             conn, addr = s.accept()
153             manejar_cliente(conn, addr)
154     except KeyboardInterrupt:
155         print("\n[servidor_contenido] Servidor detenido correctamente.")
156     finally:
157         s.close()
158
159 if __name__ == "__main__":
160     main()

```

#SIGNIFICATIVA

```
manifiesto = info.get(nombre_archivo)
if not manifiesto:
    conn.sendall(b"ERROR: Manifiesto no encontrado\n")
    return

conn.sendall(f"MANIFEST {json.dumps(manifiesto)}\n".encode())

with open(ruta_archivo, "rb") as f:
    datos_originales = f.read()

if nombre_archivo.lower().endswith((".jpg", ".jpeg", ".png")):
    datos_marcados = añadir_marca_agua_bytes(datos_originales, texto_marca=etiqueta_usuario)
else:
    datos_marcados = datos_originales

clave_b64 = manifiesto.get("key")
iv_raw = manifiesto.get("iv", "")
iv_b64 = iv_raw[4:] if iv_raw.startswith("b64:") else iv_raw

try:
    clave_bytes = base64.b64decode(clave_b64)
    iv_bytes = base64.b64decode(iv_b64)
except Exception as e:
    conn.sendall(b'{"error": "clave o iv incorrectos"}')
    print("[servidor_contenido] Error clave/iv:", e)
    return

cifrado = cifrar_aes_ctr(clave_bytes, iv_bytes, datos_marcados)

conn.sendall(f"SIZE {len(cifrado)}\n".encode())
conn.sendall(cifrado)

print(f"[servidor_contenido] Enviado {nombre_archivo} a {etiqueta_usuario}")
return

conn.sendall(b"ERROR: Comando desconocido\n")
```

Este bloque es el más significativo porque implementa el flujo principal del servidor cuando un cliente pide un archivo (comando **GET**): prepara la información necesaria, aplica protección al contenido y lo envía.

- **servidor_licencias.py**

```

1 import socket, json, base64, os
2 from utilidades_criptografia import (
3     load_rsa_private,
4     load_rsa_public,
5     rsa_decrypt_with_pow,
6     rsa_encrypt_with_pow,
7     rsa_verify_with_pow
8 )
9 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
10
11 HOST = 'localhost'
12 PORT = 6000
13
14 CLAVE_PRIVADA_SERVIDOR = 'claves/servidor_licencias_privada.pem'
15 CLAVE_PUBLICA_MODULO = 'claves/modulo_descifrado_publica.pem'
16
17 ARCHIVO_BASE_LICENCIAS = 'base_licencias_cifrada.enc'
18 CLAVE_BASE_SERVIDOR = 'claves/base_licencias_clave.bin'
19
20
21 def cargar_base_licencias(): # Descifra la base de licencias AES-GCM
22     if not os.path.exists(ARCHIVO_BASE_LICENCIAS) or not os.path.exists(CLAVE_BASE_SERVIDOR):
23         print("[servidor_licencias] No se encontró la base de datos o la clave asociada.")
24         return {}
25
26     with open(ARCHIVO_BASE_LICENCIAS, 'r', encoding='utf-8') as f:
27         contenido = json.load(f)
28
29     nonce = base64.b64decode(contenido['nonce'])
30     cifrado = base64.b64decode(contenido['ciphertext'])
31     clave = open(CLAVE_BASE_SERVIDOR, 'rb').read()
32
33     aesgcm = AESGCM(clave)
34     texto_plano = aesgcm.decrypt(nonce, cifrado, None)
35     return json.loads(texto_plano.decode())
36
37
38 def manejar_conexion(conn): # Procesa solicitud de licencia de la UA
39     data = conn.recv(8192)
40     if not data:
41         conn.close()
42         return
43
44     try:
45         solicitud = json.loads(data.decode())
46     except Exception:
47         conn.sendall(b'{"error": "JSON de solicitud invalido"}')
48         conn.close()
49         return
50
51     clave_sesion_cifrada = solicitud.get("session_key_encrypted_b64")
52     solicitud_firmada = solicitud.get("signed_request")
53
54     if not clave_sesion_cifrada or not solicitud_firmada:
55         conn.sendall(b'{"error": "faltan campos en la solicitud"}')
56         conn.close()
57         return
58
59     # Descifrar clave de sesión AES
60     try:
61         clave_privada_servidor = load_rsa_private(CLAVE_PRIVADA_SERVIDOR)
62         clave_sesion = rsa_decrypt_with_pow(clave_privada_servidor, clave_sesion_cifrada)
63     except Exception:
64         conn.sendall(b'{"error": "fallo al descifrar la clave de sesion"}')
65         conn.close()
66         return
67
68     payload_json = solicitud_firmada.get("payload_json")
69     firma_b64 = solicitud_firmada.get("signature_b64")
70
71     if not payload_json or not firma_b64:
72         conn.sendall(b'{"error": "solicitud firmada malformada"}')
73         conn.close()
74         return
75
76     # Cargar clave pública real del módulo
77     try:
78         clave_publica_modulo = load_rsa_public(CLAVE_PUBLICA_MODULO)
79     except Exception:
80         conn.sendall(b'{"error": "no se puede cargar la clave publica del modulo"}')
81         conn.close()
82         return

```

```

# Verificar firma del módulo
if not rsa_verify_with_pow(clave_publica_modulo, payload_json.encode(), firma_b64):
    conn.sendall(b'{"error": "firma digital invalida"}')
    conn.close()
    return

payload = json.loads(payload_json)
identificador_clave = payload.get("key_id")

if not identificador_clave:
    conn.sendall(b'{"error": "no se encuentra key_id en la solicitud"}')
    conn.close()
    return

# Cargar BD licencias
base_licencias = cargar_base_licencias()
if identificador_clave not in base_licencias:
    conn.sendall(b'{"error": "clave no encontrada en la base de datos"}')
    conn.close()
    return

clave_b64 = base_licencias[identificador_clave]
datos_para_modulo = json.dumps({
    "key_b64": clave_b64,
    "key_id": identificador_clave
}).encode()

# Cifrar con RSA para el módulo
try:
    datos_cifrados_b64 = rsa_encrypt_with_pow(clave_publica_modulo, datos_para_modulo)
except Exception:
    conn.sendall(b'{"error": "error cifrando los datos para el modulo"}')
    conn.close()
    return

# Envolver con AES-GCM usando la clave de sesión
aesgcm = AESGCM(clave_sesion)
nonce = os.urandom(12)
datos_envueltos = aesgcm.encrypt(nonce, datos_cifrados_b64.encode(), None)

respuesta = {
    "nonce": base64.b64encode(nonce).decode(),
    "ciphertext": base64.b64encode(datos_envueltos).decode()
}

conn.sendall(json.dumps(respuesta).encode('utf-8'))
conn.close()

def main(): # Servidor de licencias TCP
    if not os.path.exists(CLAVE_PRIVADA_SERVIDOR):
        print("[servidor_licencias] Falta la clave privada. Ejecuta generar_claves.py.")
        return

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((HOST, PORT))
    s.listen(5)
    print(f"[servidor_licencias] Escuchando en {HOST}:{PORT}")

    try:
        while True:
            conn, _ = s.accept()
            manejar_conexion(conn)
    except KeyboardInterrupt:
        print("\n[servidor_licencias] Servidor detenido correctamente.")
    finally:
        s.close()

if __name__ == "__main__":
    main()

```


Descifrar clave de sesión AES (enviada cifrada con RSA

Verificar firma del módulo (autenticidad e integridad de la solicitud)

Cifrar la respuesta para el módulo y envolverla con AES-GCM usando la clave de

sesión Este fragmento implementa la seguridad principal del servidor de licencias:

descifra la clave de sesión con RSA, verifica que la solicitud sea auténtica mediante firma digital, cifra la respuesta para el módulo y la protege con AES-GCM para enviarla de forma segura.

• modulo_descifrado.py

```
1 import socket, json, os
2 from utilidades_criptografia import (
3     load_rsa_private,
4     rsa_sign_with_pow,
5     rsa_decrypt_with_pow
6 )
7
8 HOST = 'localhost'
9 PUERTO = 7000
10 CLAVE_PRIVADA_MODULO = 'claves/modulo_descifrado_privada.pem'
11 CLAVE_PUBLICA_MODULO = 'claves/modulo_descifrado_publica.pem'
12
13 def manejar_cliente(conexion): # Gestiona las solicitudes enviadas por la aplicación de usuario
14     datos = conexion.recv(8192)
15     if not datos:
16         conexion.close()
17         return
18
19     try:
20         solicitud = json.loads(datos.decode('utf-8'))
21     except Exception:
22         conexion.sendall(json.dumps({"error": "JSON inválido"}).encode('utf-8'))
23         conexion.close()
24         return
25
26     comando = solicitud.get("cmd")
27
28     # 1. Firmar solicitud de licencia
29     if comando == "sign_request":
30         id_clave = solicitud.get("key_id")
31         id_cliente = solicitud.get("client_id")
32
33         if not id_clave or not id_cliente:
34             conexion.sendall(json.dumps({"error": "faltan campos en la solicitud"}).encode('utf-8'))
35             conexion.close()
36             return
37
38         payload = {"key_id": id_clave, "client_id": id_cliente}
39         payload_json = json.dumps(payload)
40
41         clave_privada = load_rsa_private(CLAVE_PRIVADA_MODULO)
42         firma_b64 = rsa_sign_with_pow(clave_privada, payload_json.encode('utf-8'))
43
44         respuesta = {
45             "payload_json": payload_json,
46             "signature_b64": firma_b64,
47             "cdm_pubfile": CLAVE_PUBLICA_MODULO
48         }
49
50     conexion.sendall(json.dumps(respuesta).encode('utf-8'))
51
```

```

52 # 2. Descifrar licencia enviada por la aplicación de usuario
53 elif comando == "unwrap_license":
54     licencia_cifrada_b64 = solicitud.get("encrypted_for_cdm_b64")
55
56     if not licencia_cifrada_b64:
57         conexion.sendall(json.dumps({"error": "falta encrypted_for_cdm_b64"}).encode('utf-8'))
58         conexion.close()
59         return
60
61     clave_privada = load_rsa_private(CLAVE_PRIVADA_MODULO)
62     try:
63         texto_plano = rsa_decrypt_with_pow(clave_privada, licencia_cifrada_b64)
64         conexion.sendall(texto_plano)
65     except Exception:
66         conexion.sendall(json.dumps({"error": "no se pudo descifrar la licencia en el módulo"}).encode('utf-8'))
67
68     else:
69         conexion.sendall(json.dumps({"error": "comando desconocido"}).encode('utf-8'))
70
71     conexion.close()
72
73 def main():
74     if not os.path.exists(CLAVE_PRIVADA_MODULO):
75         print("[Módulo Descifrado] Falta la clave privada. Ejecuta generar_claves.py.")
76         return
77
78     servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
79     servidor.bind((HOST, PUERTO))
80     servidor.listen(5)
81     print(f"[Módulo Descifrado] Escuchando en {HOST}:{PUERTO}")
82
83     try:
84         while True:
85             conexion, _ = servidor.accept()
86             manejar_cliente(conexion)
87     except KeyboardInterrupt:
88         print("\n[Módulo Descifrado] Servidor detenido correctamente.")
89     finally:
90         servidor.close()
91
92 if __name__ == "__main__":
93     main()

```

Firmar petición de licencia

Descifrar licencia recibida

Aquí está lo importante del módulo: firma la solicitud de licencia con su clave privada (para que el servidor verifique que es legítima) y luego descifra la licencia que le envían (porque solo este módulo, con su clave privada, puede abrirla).

• **aplicacion_usuario.py**

```

1 import socket, json, base64, os
2 from utilidades_criptografia import load_rsa_public, rsa_encrypt_with_pow
3 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
4
5 SERVIDOR CONTENIDOS_HOST = 'localhost'
6 SERVIDOR CONTENIDOS_PUERTO = 5000
7 SERVIDOR LICENCIAS_HOST = 'localhost'
8 SERVIDOR LICENCIAS_PUERTO = 6000
9 MODULO_DESCIFRADO_HOST = 'localhost'
10 MODULO_DESCIFRADO_PUERTO = 7000
11
12 CLAVE_PUBLICA_SERVIDOR = 'claves/servidor_licencias_publica.pem'
13
14 CLAVE_PUBLICA_CDM = 'claves/modulo_descifrado_publica.pem'
15
16
17 def enviar_a_servidor_contenidos(mensaje: str) -> bytes: #Envia un mensaje al servidor de contenidos y devuelve la respuesta en bytes
18     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
19     s.connect((SERVIDOR CONTENIDOS_HOST, SERVIDOR CONTENIDOS_PUERTO))
20     s.sendall(mensaje.encode('utf-8'))
21     respuesta = b''
22     while True:
23         parte = s.recv(4096)
24         if not parte:
25             break
26         respuesta += parte
27     s.close()
28     return respuesta
29
30 def contactar_cdm_firma(key_id, client_id): #Pide al módulo de descifrado (CDM) que firme una solicitud de licencia
31     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
32     s.connect((MODULO_DESCIFRADO_HOST, MODULO_DESCIFRADO_PUERTO))
33     solicitud = {"cmd": "sign_request", "key_id": key_id, "client_id": client_id}
34     s.sendall(json.dumps(solicitud).encode('utf-8'))
35     datos = s.recv(8192)
36     s.close()
37     return json.loads(datos.decode('utf-8'))
38
39 def contactar_cdm_descifrar(encrypted_for_cdm_b64): #Envia al CDM la licencia cifrada para que la descifre con su clave privada RSA
40     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
41     s.connect((MODULO_DESCIFRADO_HOST, MODULO_DESCIFRADO_PUERTO))
42     solicitud = {"cmd": "unwrap_license", "encrypted_for_cdm_b64": encrypted_for_cdm_b64}
43     s.sendall(json.dumps(solicitud).encode('utf-8'))
44     datos = s.recv(8192)
45     s.close()
46     return datos # bytes JSON con {"key_b64": "...", "key_id": "..."}
47
48 def solicitar_licencia_al_servidor(solicitud_firmada, clave_sesion): #Envia la solicitud firmada al servidor de licencias, con la clave de sesión cifrada RSA
49     server_pub = load_rsa_public(CLAVE_PUBLICA_SERVIDOR)
50     clave_sesion_cifrada_b64 = rsa_encrypt_with_pow(server_pub, clave_sesion)
51     payload = {
52         "session_key_encrypted_b64": clave_sesion_cifrada_b64,
53         "signed_request": solicitud_firmada
54     }
55     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
56     s.connect((SERVIDOR LICENCIAS_HOST, SERVIDOR LICENCIAS_PUERTO))
57     s.sendall(json.dumps(payload).encode('utf-8'))
58     respuesta = b''
59     while True:
60         parte = s.recv(8192)
61         if not parte:
62             break
63         respuesta += parte
64     s.close()
65     return json.loads(respuesta.decode('utf-8'))
66
67 # Funciones de análisis y decodificación
68
69 def parsear_manifiesto_y_resto(respuesta_bytes): #Extrae el MANIFEST JSON y devuelve (manifest_dict, resto_bytes)
70     if b"\n" not in respuesta_bytes:
71         raise ValueError("Respuesta inesperada del servidor (sin MANIFEST).")
72     primera_linea, resto = respuesta_bytes.split(b"\n", 1)
73     if not primera_linea.startswith(b"MANIFEST "):
74         raise ValueError("Respuesta sin cabecera MANIFEST.")
75     manifiesto = json.loads(primera_linea[len(b"MANIFEST "):]).decode('utf-8')
76     return manifiesto, resto
77
78 def parsear_tamano_y_datos(resto: bytes): #Extrae 'SIZE n' y devuelve los bytes exactos de datos
79     if not resto.startswith(b"SIZE "):
80         raise ValueError("Cabecera SIZE ausente en la respuesta.")
81     try:
82         linea_tamano, datos_restantes = resto.split(b"\n", 1)

```



```

81     try:
82         linea_tamano, datos_restantes = resto.split(b"\n", 1)
83     except ValueError:
84         raise ValueError("Cabecera SIZE mal formada.")
85     partes = linea_tamano.split()
86     if len(partes) < 2:
87         raise ValueError("Cabecera SIZE sin tamaño.")
88     tamano = int(partes[1])
89     if len(datos_restantes) < tamano:
90         raise ValueError(f"Datos incompletos: esperaba {tamano}, recibí {len(datos_restantes)} bytes.")
91     return datos_restantes[:tamano]
92
93 def decodificar_b64_seguro(s: str) -> bytes: #Intenta decodificar Base64 de forma segura
94     try:
95         return base64.b64decode(s)
96     except Exception:
97         raise ValueError("Cadena Base64 inválida.")
98
99
100
101 def main():
102     print("Aplicación de usuario: LIST desde el servidor de contenidos")
103     try:
104         print(enviar_a_servidor_contenidos("LIST").decode('utf-8'))
105     except Exception as e:
106         print("Error conectando al servidor de contenidos:", e)
107     return
108
109 nombre_archivo = input("Introduce el fichero a GET (ej: foto.jpg): ").strip()
110 try:
111     respuesta = enviar_a_servidor_contenidos(f"GET {nombre_archivo}")
112 except Exception as e:
113     print("Error en GET:", e)
114     return
115
116 try:
117     manifiesto, resto = parsear_manifiesto_y_resto(respuesta)
118 except Exception as e:
119     print("NO", e)
120     return
121
122 print("Manifiesto:", manifiesto)
123
124 # Si el contenido no está cifrado
125 if not manifiesto.get('encrypted', False):
126     try:
127         datos = parsear_tamano_y_datos(resto)
128         os.makedirs("descargados", exist_ok=True)
129         salida = os.path.join("descargados", nombre_archivo)
130         with open(salida, 'wb') as f:
131             f.write(datos)
132         print("Fichero descargado sin cifrado:", salida)
133     except Exception as e:
134         print("Error al guardar contenido no cifrado:", e)
135     return
136
137 # Si el contenido está cifrado
138 key_id = manifiesto.get("key_id")
139 if not key_id:
140     print("Manifiesto sin 'key_id'.")
141     return
142
143 # 1. Pedir al CDM que firme la solicitud de licencia
144 client_id = "ua-1234"
145 try:
146     solicitud_firmada = contactar_cdm_firma(key_id, client_id)
147 except Exception as e:
148     print("Error contactando al CDM:", e)
149     return
150
151 # 2. Generar clave de sesión AES (128 bits)
152 clave_sesion = AESGCM.generate_key(bit_length=128)
153
154 # 3. Solicitar licencia al servidor de licencias
155 try:
156     respuesta_envuelta = solicitar_licencia_al_servidor(solicitud_firmada, clave_sesion)
157 except Exception as e:
158     print("Error contactando con el servidor de licencias:", e)
159     return
160
161 if 'error' in respuesta_envuelta:
162     print("Error del servidor de licencias:", respuesta_envuelta)
163
164

```

```

165 # 4. Descifrar la respuesta con AES-GCM
166 nonce_b64 = respuesta_envuelta.get("nonce")
167 ct_b64 = respuesta_envuelta.get("ciphertext")
168 if not nonce_b64 or not ct_b64:
169     print("Respuesta mal formada del servidor de licencias.")
170     return
171
172 aesgcm = AESGCM(clave_sesion)
173 try:
174     datos_envueltos = aesgcm.decrypt(
175         base64.b64decode(nonce_b64),
176         base64.b64decode(ct_b64),
177         None
178     )
179 except Exception as e:
180     print("Error descifrando respuesta del servidor de licencias:", e)
181     return
182
183 # 5. Enviar licencia cifrada al CDM para descifrarla
184 encrypted_for_cdm_b64 = datos_envueltos.decode('utf-8')
185 try:
186     licencia_plana = contactar_cdm_descifrar(encrypted_for_cdm_b64)
187     licencia = json.loads(licencia_plana.decode('utf-8'))
188 except Exception as e:
189     print("Error procesando licencia del CDM:", e)
190     return
191
192 print("Licencia obtenida:", licencia)
193
194 # 6. Descifrar el contenido
195 key_b64 = licencia.get('key_b64')
196 if not key_b64:
197     print("Licencia sin clave (key_b64).")
198     return
199
200 clave = base64.b64decode(key_b64)
201 iv_campo = manifiesto.get("iv")
202 if not iv_campo:
203     print("IV ausente en el manifiesto.")
204     return
205
206 iv_b64 = iv_campo[4:] if iv_campo.startswith("b64:") else iv_campo
207 iv = decodificar_b64_seguro(iv_b64)
208
209 try:
210     datos_cifrados = parsear_tamano_y_datos(resto)
211 except Exception as e:
212     print("Error obteniendo datos cifrados:", e)
213     return
214
215 modo = manifiesto.get("mode", "CTR").upper()
216 from utilidades_criptografia import aes_decrypt_content_ctr, aes_decrypt_content_cbc
217 try:
218     if modo == "CTR":
219         plano = aes_decrypt_content_ctr(clave, iv, datos_cifrados)
220     elif modo == "CBC":
221         plano = aes_decrypt_content_cbc(clave, iv, datos_cifrados)
222     else:
223         print("Modo de cifrado no soportado:", modo)
224         return
225 except Exception as e:
226     print("Error al descifrar el contenido:", e)
227     return
228
229 os.makedirs("descifrados", exist_ok=True)
230 salida_final = os.path.join("descifrados", nombre_archivo)
231 with open(salida_final, 'wb') as f:
232     f.write(plano)
233 print("Contenido descifrado guardado en", salida_final)
234
235 if __name__ == "__main__":
236     main()

```

1) El CDM firma la solicitud de licencia

2) Se crea una clave de sesión y se pide la licencia al servidor (clave de sesión cifrada con RSA)

3) Se abre la respuesta con AES-GCM y el CDM descifra la licencia (RSA)

4) Con la clave de la licencia se descifra el contenido descargado (AES CTR/CBC)
Este bloque es el núcleo: pide una licencia (con solicitud firmada + clave de sesión),
la descifra (AES-GCM + RSA vía CDM) y usa la clave obtenida para descifrar el
contenido descargado (AES CTR/CBC).

• utilidades_criptografia.py

```
1 import os, base64
2 from hashlib import sha256
3 from cryptography.hazmat.primitives import serialization as serializacion
4 from cryptography.hazmat.primitives import padding as relleno_pkcs7
5 from cryptography.hazmat.primitives.asymmetric import rsa as rsa_criptografia
6 from cryptography.hazmat.primitives.ciphers import Cipher as Cifrado, algorithms as algoritmos, modes as modos
7 from cryptography.hazmat.backends import default_backend
8 from cryptography.hazmat.primitives.ciphers.aead import AESGCM as AESGCM_
9
10
11 def generar_par_rsa(tamano_clave=2048):
12     clave_privada = rsa_criptografia.generate_private_key(public_exponent=65537, key_size=tamano_clave)
13     clave_publica = clave_privada.public_key()
14     return clave_privada, clave_publica
15
16 def guardar_rsa_privada(clave_privada, ruta):
17     pem = clave_privada.private_bytes(
18         encoding=serializacion.Encoding.PEM,
19         format=serializacion.PrivateFormat.PKCS8,
20         encryption_algorithm=serializacion.NoEncryption()
21     )
22     with open(ruta, 'wb') as f:
23         f.write(pem)
24
25 def guardar_rsa_publica(clave_publica, ruta):
26     pem = clave_publica.public_bytes(
27         encoding=serializacion.Encoding.PEM,
28         format=serializacion.PublicFormat.SubjectPublicKeyInfo
29     )
30     with open(ruta, 'wb') as f:
31         f.write(pem)
32
33 def load_rsa_private(ruta):
34     with open(ruta, 'rb') as f:
35         return serializacion.load_pem_private_key(f.read(), password=None)
36
37 def load_rsa_public(ruta):
38     with open(ruta, 'rb') as f:
39         return serializacion.load_pem_public_key(f.read())
40
41 def rsa_numeros_desde_privada(clave_privada):
42     nums = clave_privada.private_numbers()
43     d = nums.d
44     n = nums.public_numbers.n
45     e = nums.public_numbers.e
46     return (n, e, d)
47
48 def rsa_numeros_desde_publica(clave_publica):
49     nums = clave_publica.public_numbers()
50     return (nums.n, nums.e)
51
52 def bytes_a_entero(b: bytes) -> int:
53     return int.from_bytes(b, byteorder='big')
54
55 def entero_a_bytes(i: int, longitud_minima=0) -> bytes:
56     longitud = max((i.bit_length() + 7) // 8, longitud_minima)
57     return i.to_bytes(longitud, byteorder='big')
58
59 def rsa_encrypt_with_pow(clave_publica, datos: bytes) -> str:
60     n, e = rsa_numeros_desde_publica(clave_publica)
61     m = bytes_a_entero(datos)
62     if m >= n:
63         raise ValueError("Mensaje demasiado largo para RSA.")
64     c = pow(m, e, n)
65     cb = entero_a_bytes(c)
66     return base64.b64encode(cb).decode()
67
```

```

68 def rsa_decrypt_with_pow(clave_privada, cifrado_b64: str) -> bytes:
69     n, e, d = rsa_numeros_desde_privada(clave_privada)
70     cb = base64.b64decode(cifrado_b64)
71     c = bytes_a_entero(cb)
72     m = pow(c, d, n)
73     return entero_a_bytes(m)
74
75 def rsa_sign_with_pow(clave_privada, mensaje: bytes) -> str:
76     h = sha256(mensaje).digest()
77     m = bytes_a_entero(h)
78     n, e, d = rsa_numeros_desde_privada(clave_privada)
79     s = pow(m, d, n)
80     return base64.b64encode(entero_a_bytes(s)).decode()
81
82 def rsa_verify_with_pow(clave_publica, mensaje: bytes, firma_b64: str) -> bool:
83     h = sha256(mensaje).digest()
84     m = bytes_a_entero(h)
85     n, e = rsa_numeros_desde_publica(clave_publica)
86     s = bytes_a_entero(base64.b64decode(firma_b64))
87     v = pow(s, e, n)
88     return v == m
89
90 def aesgcm_cifrar(clave: bytes, datos: bytes, aad: bytes = b''):
91     aesgcm = AESGCM(clave)
92     nonce = os.urandom(12)
93     ct = aesgcm.encrypt(nonce, datos, aad)
94     return {
95         "nonce": base64.b64encode(nonce).decode(),
96         "ciphertext": base64.b64encode(ct).decode()
97     }
98
99 def aesgcm_descifrar(clave: bytes, nonce_b64: str, ct_b64: str, aad: bytes = b''):
100     aesgcm = AESGCM(clave)
101     nonce = base64.b64decode(nonce_b64)
102     ct = base64.b64decode(ct_b64)
103     return aesgcm.decrypt(nonce, ct, aad)
104
105 def aes_cifrar_contenido_ctr(clave: bytes, iv: bytes, datos: bytes) -> bytes:
106     cifrador = Cifrado(algoritmos.AES(clave), modos.CTR(iv), backend=default_backend()).encryptor()
107     return cifrador.update(datos) + cifrador.finalize()
108
109 def aes_descifrar_contenido_ctr(clave: bytes, iv: bytes, datos_cifrados: bytes) -> bytes:
110     descifrador = Cifrado(algoritmos.AES(clave), modos.CTR(iv), backend=default_backend()).decryptor()
111     return descifrador.update(datos_cifrados) + descifrador.finalize()
112
113 def aes_cifrar_contenido_cbc(clave: bytes, iv: bytes, datos: bytes) -> bytes:
114     padder = rellenado_pkcs7.PKCS7(128).padder()
115     padded = padder.update(datos) + padder.finalize()
116     cifrador = Cifrado(algoritmos.AES(clave), modos.CBC(iv), backend=default_backend()).encryptor()
117     return cifrador.update(padded) + cifrador.finalize()
118
119 def aes_descifrar_contenido_cbc(clave: bytes, iv: bytes, datos_cifrados: bytes) -> bytes:
120     descifrador = Cifrado(algoritmos.AES(clave), modos.CBC(iv), backend=default_backend()).decryptor()
121     padded = descifrador.update(datos_cifrados) + descifrador.finalize()
122     unpadder = rellenado_pkcs7.PKCS7(128).unpadder()
123     return unpadder.update(padded) + unpadder.finalize()

```

#Significatvo

```

59 def rsa_encrypt_with_pow(clave_publica, datos: bytes) -> str:
60     n, e = rsa_numeros_desde_publica(clave_publica)
61     m = bytes_a_entero(datos)
62     if m >= n:
63         raise ValueError("Mensaje demasiado largo para RSA.")
64     c = pow(m, e, n)
65     cb = entero_a_bytes(c)
66     return base64.b64encode(cb).decode()
67
68 def rsa_decrypt_with_pow(clave_privada, cifrado_b64: str) -> bytes:
69     n, e, d = rsa_numeros_desde_privada(clave_privada)
70     cb = base64.b64decode(cifrado_b64)
71     c = bytes_a_entero(cb)
72     m = pow(c, d, n)
73     return entero_a_bytes(m)
74

```


Aquí está lo clave del archivo: implementa RSA “a mano” usando `pow(m, e, n)` para cifrar y `pow(c, d, n)` para descifrar (exponenciación modular). Convierte bytes↔entero para aplicar RSA y usa Base64 para poder transportar el resultado como texto.

6. Componentes del Sistema (en orden de ejecución)

- `generar_claves.py` — crea claves RSA para servidor y CDM.
- `generar_base_licencias.py` — genera base cifrada con claves AES.
- `cifrar_contenido.py` — aplica marca de agua y cifra contenidos para uso del servidor.
- `contenido_servidor.py` — envía contenidos cifrados y manifiestos.
- `servidor_licencias.py` — gestiona claves, verifica firmas y envía licencias.
- `modulo_descifrado.py` — firma solicitudes y descifra licencias.
- `aplicacion_usuario.py` — coordina toda la comunicación.
- `utilidades_criptografia.py` — RSA, AES, firmas, padding, etc.

7. Capturas de Funcionamiento

1. Claves

```
>>> %Run generar_claves.py
Claves RSA generadas para 'servidor_licencias' en claves/servidor_licencias_*.pem
Claves RSA generadas para 'modulo_descifrado' en claves/modulo_descifrado_*.pem
>>>
```

2. Base licencias

```
>>> %Run generar_base_licencias.py
[generar_base_licencias] Clave maestra ya existente, se reutiliza.
[generar_base_licencias] Base de datos cifrada creada correctamente.
>>>
```

3. Cifrar contenido

```
>>> %Run cifrar_contenido.py

Añadiendo marca de agua a la_falete.jpg...
'la_falete.jpg' cifrado correctamente y guardado en contenido_cifrado\la_falete.jpg
Añadiendo marca de agua a rioja.jpg...
'rioja.jpg' cifrado correctamente y guardado en contenido_cifrado\rioja.jpg
'torrente.mp4' cifrado correctamente y guardado en contenido_cifrado\torrente.mp4
'remontada.mp4' cifrado correctamente y guardado en contenido_cifrado\remontada.mp4

>>>
```

4. Contenido servidor

```
>>> %Run contenido_servidor.py

[servidor_contenido] Escuchando en localhost:5000
```

5. Servidor licencias

```
>>> %Run servidor_licencias.py

[servidor_licencias] Escuchando en localhost:6000
```

6. Módulo Descifrado

```
>>> %Run modulo_descifrado.py

[Módulo Descifrado] Escuchando en localhost:7000
```

7. Aplicacion usuario


```
>>> %Run aplicacion_usuario.py

Aplicación de usuario: LIST desde el servidor de contenidos
la_falete.jpg
remontada.mp4
rioja.jpg
torrente.mp4
END

Introduce el fichero a GET (ej: foto.jpg): rioja.jpg
Manifiesto: {'encrypted': True, 'mode': 'CTR', 'key': 'dK5xPzQ7w9kK89jsAlX9tg==', 'iv': 'b64:Gc7Pw92B7Z6eMlpE8sLxVg==', 'key_id': 'key_rioja'}
Licencia obtenida: {'key_b64': 'dK5xPzQ7w9kK89jsAlX9tg==', 'key_id': 'key_rioja'}
Contenido descifrado guardado en descifrados\rioja.jpg

>>>
```

8. Visualización del archivo descifrado con marca de agua



8. Pruebas y Validación

Las pruebas realizadas incluyen:

- Verificación de comunicación correcta entre todos los módulos.
- Comprobación de cifrado AES-CTR y descifrado correcto.
- Test de firma RSA y validación en servidor.
- Comprobación de que cada licencia se entrega solo si la firma es válida.
- Verificación visual de la marca de agua en imágenes.